

Name - Xisha Bhaumik

Roll no. - VC/04/BTCE/2025/001

1. (b) Data and a pointer to the next node
2. (c) NULL
3. (b) Self-referential structure
4. (b) $head == NULL$
5. (b) $newnode \rightarrow next = head; head = newnode;$
6. (b) $temp = head; head = head \rightarrow next;$
 $free(temp);$
7. (a) Traversing from head to tail
8. (b) n
9. (c) Copying the data of $P \rightarrow next$ into P ,
then deleting $P \rightarrow next$
10. (b) $while (P != NULL)$
11. (b) $P \rightarrow next == NULL$
12. (c) Two pointers advancing at different rates
13. (c) Three
14. (b) Exactly one node
15. (b) $P = (struct node*) malloc(sizeof(struct node));$
16. (b) Avoid special-case handling for insertion and deletion at the head
17. (c) The first node
18. (b) It does not support random (index) access
19. (b) Loss of access to all nodes, causing a memory leak.

20. (b) A slow pointers and a fast pointer

21. (c) The last node

22. (b) Nodes may be scattered anywhere in memory. side []

```
23. #include <stdio.h>
#include <stdio.h>
struct Node {
    int data;
    struct Node *next;
};
```

24. The code prints the sum of each node and its next node:

$$\rightarrow 10 + 20 = 30$$

$$\rightarrow 20 + 30 = 50$$

$$\rightarrow 30 + 40 = 70$$

output is $\rightarrow 30 \quad 50 \quad 70$

```
25. void insertEnd (struct node * head, int x) {
```

```
    struct node = t = (struct node) malloc
    (size of (struct node));
```

```
    t  $\rightarrow$  data = x;
```

```
    t  $\rightarrow$  next = null;
```

```
    if (*head == null) {
```

```
*head = t;  
return;
```

```
}
```

```
struct node* p = *head  
while (p->next != NULL)
```

```
    p = p->next;  
    p->next = t;
```

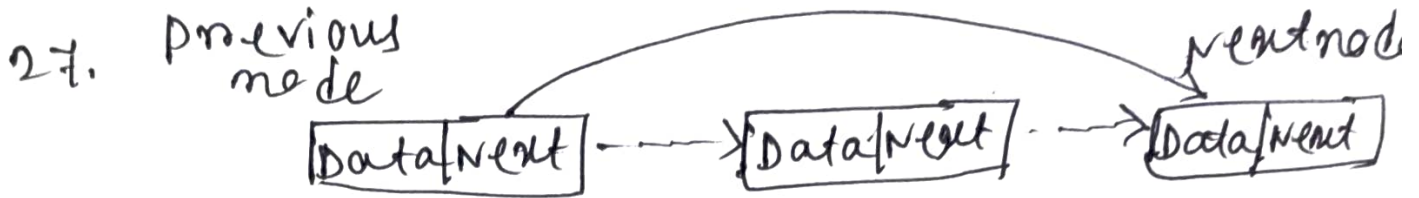
```
}
```

```
26. struct node * reverse  
    (struct node * head) {  
    struct node * prev = NULL;  
    struct node * curr = head;  
    struct node * next = NULL;  
    while (curr != NULL) {  
        next = curr->next;  
        curr->next = prev;  
        prev = curr;  
        curr = next;
```

```
}
```

```
    return prev;
```

```
}
```



pointers adjustment
 $prev \rightarrow next \rightarrow del \rightarrow next;$
 $free(del)$

28. $5 \rightarrow 8 \rightarrow 12 \rightarrow 3 \rightarrow \text{Null}$
 $7 \rightarrow 5 \rightarrow 8 \rightarrow 12 \rightarrow 3 \rightarrow \text{Null}$
 $7 \rightarrow 5 \rightarrow 8 \rightarrow 3 \rightarrow 9 \rightarrow \text{Null}$
 Final $\rightarrow 7 \rightarrow 5 \rightarrow 8 \rightarrow 3 \rightarrow 9 \rightarrow \text{Null}$

Data structure and Algorithms
 Zisha Bhaumik
 NGE/04/BTCE/2025/001